CTU training solutions | Private Tertiary Education

CAPSTONE PROJECT

CHAPTER 2 - LITERATURE REVIEW

Student / Group Template

Project Title	Customer Support Ticketing System: Design and Development of a Web-Based Support Request Management Application
Group Members	Freek Schaefer, Wian Sandenbergh
Module	SPM622 / MDB622
Lecturer / Supervisor	Mr. Dunstan
Week / Stage	Literature Review finalised
Submission Date	29-10-2026

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07

2.1 Introduction

Explain what this chapter reviews and how it supports the project.

This chapter reviews approaches to recording, assigning and resolving customer service requests and examines established ticketing products. It considers system functionality, implementation choices and limitations relevant to a small-team academic prototype. The review is based primarily on published product and technical documentation; it does not constitute an empirical study of customer satisfaction or system performance.

2.2 Existing Research

Review credible academic and industry sources related to the problem.

A customer support ticket is a structured record of a request, commonly including a requester, description, status, priority and assigned agent. Zendesk documentation identifies these as standard ticket fields and explains how they support organisation and workflow rules (Zendesk, 2026a). Its ticket API also distinguishes the customer-facing request from the agent-facing ticket, illustrating why access rights must depend on user role (Zendesk, n.d.). These practices motivate the project's ticket model, customer portal and role-based permissions.

Industry documentation also demonstrates the importance of a shared queue and ticket history. osTicket documents a multi-user interface that consolidates requests from several channels and supports a customer portal (osTicket, n.d.a). For the proposed prototype, a central record and clear ownership are more feasible than comprehensive omnichannel integration. The available product documents explain capabilities but do not independently establish that a particular product improves performance in every small organisation; that outcome would require separate empirical evaluation.

2.3 Existing or Similar Systems

Compare at least three relevant systems. Discuss features, strengths, limitations and lessons for your solution.

Zendesk Support is a mature ticketing platform with configurable fields, multiple support channels, agent workspaces, reporting and automation (Zendesk, 2026b). Its breadth is useful as a functional benchmark but exceeds the intended scope of a student-built minimum viable product.

Freshdesk offers ticket management, shared inboxes, customer portals, routing, service-level features and analytics (Freshworks, n.d.). These features provide a useful reference for prioritisation and dashboards. The project will not attempt to replicate its full automation and AI functionality.

osTicket is an open-source helpdesk with configurable ticket forms, agent assignment, queues, a customer portal and service-level features (osTicket, n.d.b). Its web-based ticket lifecycle and straightforward role separation are particularly relevant to the proposed prototype. Its breadth of administrative settings is beyond the immediate project scope.

The comparison shows common baseline functions: ticket submission, ownership, status tracking and conversation history. Our project will implement a selected subset and evaluate it through reproducible tests, rather than claiming to outperform these established products.

2.4 Technology Review

Evaluate technologies, frameworks, databases, cloud platforms or tools relevant to the project.

The final implementation of the SmartDesk prototype utilises a Python-based backend using the Flask web framework and Jinja2 for server-side HTML rendering. A SQLite relational database was selected for data storage, providing a lightweight, zero-configuration database that is ideal for a small-team MVP and local demonstration. SQLite handles users, tickets, comments, and status changes. Python's sqlite3 standard library was used for parameterised database access to prevent SQL injection.

Git and GitHub were used to support collaboration, branch management, and evidence of contributions.

Input validation, password hashing (via Werkzeug), server-side authorisation, and CSRF protection were implemented as required design considerations across the stack.

2.5 Research Gap

Identify what existing research or systems do not adequately solve. State the gap your project addresses.

The identified gap is not a lack of customer support software: several mature platforms already exist. Instead, this capstone addresses a narrower educational and practical need: a transparent, small-scale, customisable ticket workflow that a two-person team can design, implement, explain and test end to end without relying on a commercial helpdesk as the core application. The project will focus on essential ticket lifecycle functions, explicit database relationships and traceable testing evidence. This is a project-specific scope gap, not a claim of novel scientific discovery.

2.6 Conceptual / Technical Direction

Explain the key ideas or technologies that guide your proposed solution.

The implemented workflow is as follows: a customer registers or signs in; creates a ticket with a category, subject, and description; the system records a unique reference and initial status; an administrator or agent assigns and prioritises the ticket; the customer and agent exchange comments; the agent marks the request as resolved; and the ticket is closed after review.

Three user roles were implemented: Customer (create and view own tickets), Agent (manage assigned tickets and reply), and Administrator (manage assignments, users, and reports). The core database entities implemented include User, Ticket, and Comment. Functional acceptance tests covered ticket creation, visibility restrictions, assignment, comments, and state changes; non-functional checks covered usability, input validation, and basic security.

tests will cover ticket creation, visibility restrictions, assignment, comments and state changes; non-functional checks will cover usability, validation and basic security.

2.7 Chapter Summary

Summarise the main findings and link them to requirements.

The review establishes a defensible baseline for a customer support ticketing prototype. Existing products demonstrate the value of centralised records, role-aware access, assignment and status tracking. The selected scope prioritises a complete, testable workflow over advanced features such as omnichannel messaging or AI automation. Chapter 3 will translate these findings into stakeholder needs, measurable requirements, use cases and a feasible development method.